Birzeit university

Faculty of Engineering and Technology

<u>Computer Science Department</u>

Database Systems 333

Project Report

Prepared by:

Nagham Massis - 1220149

Nooreen Murad - 1222470

Project Tittle: Women’s Handbag Store and its Warehouse Management

Instructor: Dr. Mohammad "Hussein Ali"

Section No: "5"

Date: 18/1/2026

Table of Contents

Project Idea: 1

Target Users: 1

Client and Contact Information: 1

Entities and Their Attributes: 2

Relationships 7

Sample Queries 16

Entity-Relationship Diagram (ERD): 17

Project Idea:

Our project is about creating a database system for a women's handbag management store and its warehouse.

The goal is to make it easier for the store owner to manage everything in one place like buying handbags from suppliers, storing them in the warehouse, and organizing the customer's purchase process, while the same database keeps way of all products and warehouse updates.

The system helps keep way of the available quantities, the different handbag types, and whether the warehouse is full or needs restocking.

It also provides simple reports about sales, purchases, and stock levels to make store management faster and easier.

Target Users:

This system is for store owners and employees who sell women's handbags. It helps them keep path of sales, purchases, and what's available in the warehouse in an easy and organized way.

Client and Contact Information:

Our client is Miss Layali Massis, the owner of Cherry Boutique, a women's handbag store.

She wants a system that helps her manage and show her products, sales, and warehouse more professionally.

Phone: 0597 654 321

Email: CherryBoutique@gmail.com

Entities and Their Attributes:

1) Customer: Stores details about customers.

- Customer_ID (PK)
- Name
- Phone
- Address

2) Store: Represents a store's information.

- Store_ID (PK)
- Name
- Location

3) Warehouse: Represents a warehouse's information.

- Warehouse_ID (PK)
- Location
- Capacity

4) Employee: Represents employee information.

- Employee_ID (PK)
- Name
- Role (Manager, Cashier, Warehouse Staff)

5) Store_Employee: Employees working at a store.

- Employee_ID (PK, FK)
- Store_ID (FK)
- Foreign Key Constraints:
    - References Employee (Employee_ID)
    - References Store (Store_ID)

6) Warehouse_Employee: Employees working at a warehouse.

- Employee_ID (PK, FK)
- Warehouse_ID (FK)
- Foreign Key Constraints:
    - References Employee (Employee_ID)
    - References Warehouse (Warehouse_ID)

7) Category: Product category/grouping.

- Category_ID (PK)
- Category_Name

8) Supplier: Suppliers who provide items.

- Supplier_ID (PK)
- Name
- Phone

- Email

9) Item: Product catalog (handbag items).

- Item_ID (PK)
- Item_Name
- Category_ID (FK)

10) Store_Inventory: Inventory of each item in each store (includes selling price).

- Store_ID (FK)
- Item_ID (FK)
- Selling_Price
- Quantity_Store

Primary Key (Store_ID, Item_ID)

11) Stock: Current stock of each item in each warehouse.

- Warehouse_ID (FK)
- Item_ID (FK)
- Quantity_Warehouse

Primary Key (Warehouse_ID, Item_ID)

12) Customer_Order: Customer purchase transaction at a store.

- CustomerOrder_ID (PK)
- Customer_ID (FK)

- Employee_ID (FK) (processed by)
- Store_ID (FK) (order location)
- Order_Date
- Total_Amount (Derived)

13) CustomerOrder_Details: Items included in a customer order.

- CustomerOrder_ID (FK)
- Item_ID (FK)
- Quantity
- Unit_Price

Primary Key (CustomerOrder_ID, Item_ID)

14) Supply_Order: Supplier restocking transaction delivered to a warehouse.

- SupplyOrder_ID (PK)
- Supplier_ID (FK)
- Employee_ID (FK) (created by)
- Warehouse_ID (FK) (receiving warehouse)
- Order_Date
- Total_Amount (Derived)

15) SupplyOrder_Detail: Items included in a supply order.

- SupplyOrder_ID (FK)

- Item_ID (FK)
- Quantity_Purchased
- Unit_Cost

Primary Key (SupplyOrder_ID, Item_ID)

16) Stock_Transfer (Header): Transfer operation from warehouse to store.

- Transfer_ID (PK)
- Warehouse_ID (FK) (source)
- Store_ID (FK) (destination)
- Employee_ID (FK) (performed by)
- Transfer_Date

17) StockTransfer_Details: Items transferred in a transfer.

- Transfer_ID (FK)
- Item_ID (FK)
- Quantity_Transferred

Primary Key (Transfer_ID, Item_ID)

Relationships:

R1) Customer places Customer_Order

- Relationship: Customer (1) — (M) Customer_Order
- Implemented by: Customer_Order.Customer_ID (FK)

R2) Employee processes Customer_Order

- Relationship: Employee (1) — (M) Customer_Order
- Implemented by: Customer_Order.Employee_ID (FK)

R3) Store receives Customer_Order (order is made at store)

- Relationship: Store (1) — (M) Customer_Order
- Implemented by: Customer_Order.Store_ID (FK)

R4) Customer_Order contains CustomerOrder_Details

- Relationship: Customer_Order (1) — (M) CustomerOrder_Details
- Implemented by: CustomerOrder_Details.CustomerOrder_ID (FK)

R5) Item appears in CustomerOrder_Details

- Relationship: Item (1) — (M) CustomerOrder_Details
- Implemented by: CustomerOrder_Details.Item_ID (FK)
- Meaning: Customer_Order (M) — (N) Item resolved by CustomerOrder_Details

R6) Category has Items

- Relationship: Category (1) — (M) Item
- Implemented by: Item.Category_ID (FK)

R7) Supplier provides Supply_Order

- Relationship: Supplier (1) — (M) Supply_Order
- Implemented by: Supply_Order.Supplier_ID (FK)

R8) Employee creates Supply_Order

- Relationship: Employee (1) — (M) Supply_Order
- Implemented by: Supply_Order.Employee_ID (FK)

R9) Warehouse receives Supply_Order

- Relationship: Warehouse (1) — (M) Supply_Order
- Implemented by: Supply_Order.Warehouse_ID (FK)

R10) Supply_Order contains SupplyOrder_Detail

- Relationship: Supply_Order (1) — (M) SupplyOrder_Detail
- Implemented by: SupplyOrder_Detail.SupplyOrder_ID (FK)

R11) Item included in SupplyOrder_Detail

- Relationship: Item (1) — (M) SupplyOrder_Detail
- Implemented by: SupplyOrder_Detail.Item_ID (FK)
- Meaning: Supply_Order (M) — (N) Item resolved by SupplyOrder_Detail

R12) Store stocks Items (Store_Inventory)

- Relationship: Store (1) — (M) Store_Inventory
- Implemented by: Store_Inventory.Store_ID (FK)

R13) Item is stocked in Store_Inventory

- Relationship: Item (1) — (M) Store_Inventory
- Implemented by: Store_Inventory.Item_ID (FK)
- Meaning: Store (M) — (N) Item resolved by Store_Inventory

R14) Warehouse stores Stock records

- Relationship: Warehouse (1) — (M) Stock
- Implemented by: Stock.Warehouse_ID (FK)

R15) Item exists in Stock

- Relationship: Item (1) — (M) Stock
- Implemented by: Stock.Item_ID (FK)
- Meaning: Warehouse (M) — (N) Item resolved by Stock

R16) Warehouse is the source of Stock_Transfer

- Relationship: Warehouse (1) — (M) Stock_Transfer
- Implemented by: Stock_Transfer.Warehouse_ID (FK)

R17) Store is the destination of Stock_Transfer

- Relationship: Store (1) — (M) Stock_Transfer
- Implemented by: Stock_Transfer.Store_ID (FK)

R18) Employee performs Stock_Transfer

- Relationship: Employee (1) — (M) Stock_Transfer
- Implemented by: Stock_Transfer.Employee_ID (FK)

R19) Stock_Transfer contains StockTransfer_Details

- Relationship: Stock_Transfer (1) — (M) StockTransfer_Details
- Implemented by: StockTransfer_Details.Transfer_ID (FK)

R20) Item appears in StockTransfer_Details

- Relationship: Item (1) — (M) StockTransfer_Details
- Implemented by: StockTransfer_Details.Item_ID (FK)
- Meaning: Stock_Transfer (M) — (N) Item resolved by StockTransfer_Details

Normalization to 3NF (per relation):

1) Customer

- **Attributes:** Customer_ID, Name, Phone, Address
- **Primary Key:** Customer_ID
- **Functional Dependencies:**
  - Customer_ID → Name, Phone, Address
- **Justification:**
  - **1NF:** All attributes are atomic.
  - **2NF:** No partial dependency, as Customer_ID is the only key.
  - **3NF:** No transitive dependency (non-key attributes only depend on the key).
  - **Conclusion: Satisfies 3NF.**

2) Store

- **Attributes:** Store_ID, Name, Location
- **Primary Key:** Store_ID
- **Functional Dependencies:**
  - Store_ID → Name, Location
- **Justification:**
  - **1NF:** All attributes are atomic.
  - **2NF:** Single attribute primary key, no partial dependency.
  - **3NF:** No transitive dependency.
  - **Conclusion: Satisfies 3NF.**

3) Warehouse

- **Attributes:** Warehouse_ID, Location, Capacity
- **Primary Key:** Warehouse_ID
- **Functional Dependencies:**
  - Warehouse_ID → Location, Capacity
- **Justification:**
  - **1NF:** All attributes are atomic.
  - **2NF:** Single attribute primary key, no partial dependency.
  - **3NF:** No transitive dependency.
  - **Conclusion: Satisfies 3NF.**

4) Employee

- **Attributes:** Employee_ID, Name, Role,
- **Primary Key:** Employee_ID
- **Functional Dependencies:**
  - Employee_ID → Name, Role
- **Justification:**
  - **1NF:** All attributes are atomic.
  - **2NF:** Single attribute primary key, no partial dependency.
  - **3NF:** No transitive dependency.
  - **Conclusion: Satisfies 3NF.**

5) Store_Employee

- **Attributes:** Employee_ID, Store_ID
- **Primary Key:** (Employee_ID, Store_ID)
- **Functional Dependencies:**
  - (Employee_ID, Store_ID) → Employee_ID, Store_ID
- **Justification:**
  - **1NF:** All attributes are atomic.
  - **2NF:** No partial dependencies.
  - **3NF:** No transitive dependency.
  - **Conclusion: Satisfies 3NF.**

6) Warehouse_Employee

- **Attributes:** Employee_ID, Warehouse_ID
- **Primary Key:** (Employee_ID, Warehouse_ID)
- **Functional Dependencies:**
  - (Employee_ID, Warehouse_ID) → Employee_ID, Warehouse_ID
- **Justification:**
  - **1NF:** All attributes are atomic.
  - **2NF:** No partial dependencies.
  - **3NF:** No transitive dependency.
  - **Conclusion: Satisfies 3NF.**

7) Category

- **Attributes:** Category_ID, Category_Name
- **Primary Key:** Category_ID
- **Functional Dependencies:**
  - Category_ID → Category_Name
- **Justification:**
  - **1NF:** All attributes are atomic.
  - **2NF:** Single attribute primary key, no partial dependency.
  - **3NF:** No transitive dependency.
  - **Conclusion: Satisfies 3NF.**

8) Supplier

- **Attributes:** Supplier_ID, Name, Phone, Email
- **Primary Key:** Supplier_ID
- **Functional Dependencies:**
  - Supplier_ID → Name, Phone, Email
- **Justification:**
  - **1NF:** All attributes are atomic.
  - **2NF:** Single attribute primary key, no partial dependency.
  - **3NF:** No transitive dependency.
  - **Conclusion: Satisfies 3NF.**

9) Item

- **Attributes:** Item_ID, Item_Name, Category_ID
- **Primary Key:** Item_ID
- **Functional Dependencies:**
  - Item_ID → Item_Name, Category_ID
- **Justification:**
  - **1NF:** All attributes are atomic.
  - **2NF:** No partial dependency.
  - **3NF:** No transitive dependency.
  - **Conclusion: Satisfies 3NF.**

10) Store_Inventory

- **Attributes:** Store_ID, Item_ID, Selling_Price, Quantity_Store
- **Primary Key:** (Store_ID, Item_ID)
- **Functional Dependencies:**
  - (Store_ID, Item_ID) → Selling_Price, Quantity_Store
- **Justification:**
  - **1NF:** All attributes are atomic.
  - **2NF:** Non-key attributes depend on the full composite key.
  - **3NF:** No transitive dependency.
  - **Conclusion: Satisfies 3NF.**

11) Stock

- **Attributes:** Warehouse_ID, Item_ID, Quantity_Warehouse
- **Primary Key:** (Warehouse_ID, Item_ID)
- **Functional Dependencies:**
  - (Warehouse_ID, Item_ID) → Quantity_Warehouse
- **Justification:**
  - **1NF:** All attributes are atomic.
  - **2NF:** Non-key attributes depend on the full composite key.
  - **3NF:** No transitive dependency.
  - **Conclusion: Satisfies 3NF.**

12) Customer_Order

- **Attributes:** CustomerOrder_ID, Order_Date, Customer_ID, Employee_ID, Store_ID, Total_Amount
- **Primary Key:** CustomerOrder_ID
- **Functional Dependencies:**
  - CustomerOrder_ID → Order_Date, Customer_ID, Employee_ID, Store_ID
  - Total_Amount is derived from CustomerOrder_Details (Quantity × Unit_Price).
- **Justification:**
  - **1NF:** All attributes are atomic.
  - **2NF:** Non-key attributes depend on the full key.
  - **3NF:**

- **Conclusion: Satisfies 3NF.**

13) CustomerOrder_Details

- **Attributes:** CustomerOrder_ID, Item_ID, Quantity, Unit_Price
- **Primary Key:** (CustomerOrder_ID, Item_ID)
- **Functional Dependencies:**
  - (CustomerOrder_ID, Item_ID) → Quantity, Unit_Price
- **Justification:**
  - **1NF:** All attributes are atomic.
  - **2NF:** Non-key attributes depend on the full composite key.
  - **3NF:** No transitive dependency.
  - **Conclusion: Satisfies 3NF.**

14) Supply_Order

- **Attributes:** SupplyOrder_ID, Supplier_ID, Employee_ID, Warehouse_ID, Order_Date, Total_Amount
- **Primary Key:** SupplyOrder_ID
- **Functional Dependencies:**
  - SupplyOrder_ID → Supplier_ID, Employee_ID, Warehouse_ID, Order_Date
  - Total_Amount is derived from SupplyOrder_Detail.
- **Justification:**
  - **1NF:** All attributes are atomic.
  - **2NF:** Non-key attributes depend on the full key.
  - **3NF:** No transitive dependency (Total_Amount is derived).
  - **Conclusion: Satisfies 3NF.**

15) SupplyOrder_Detail

- **Attributes:** SupplyOrder_ID, Item_ID, Quantity_Purchased
- **Primary Key:** (SupplyOrder_ID, Item_ID)
- **Functional Dependencies:**
  - (SupplyOrder_ID, Item_ID) → Quantity_Purchased
- **Justification:**
  - **1NF:** All attributes are atomic.
  - **2NF:** Non-key attributes depend on the full composite key.

    - **3NF:** No transitive dependency.
    - **Conclusion: Satisfies 3NF.**

16) Stock_Transfer

- **Attributes:** Transfer_ID, Warehouse_ID, Store_ID, Employee_ID, Transfer_Date
- **Primary Key:** Transfer_ID
- **Functional Dependencies:**
    - Transfer_ID → Warehouse_ID, Store_ID, Employee_ID, Transfer_Date
- **Justification:**
    - **1NF:** All attributes are atomic.
    - **2NF:** Non-key attributes depend on the full key.
    - **3NF:** No transitive dependency.
    - **Conclusion: Satisfies 3NF.**

17) StockTransfer_Details

- **Attributes:** Transfer_ID, Item_ID, Quantity_Transferred
- **Primary Key:** (Transfer_ID, Item_ID)
- **Functional Dependencies:**
    - (Transfer_ID, Item_ID) → Quantity_Transferred
- **Justification:**
    - **1NF:** All attributes are atomic.
    - **2NF:** Non-key attributes depend on the full composite key.
    - **3NF:** No transitive dependency.
    - **Conclusion: Satisfies 3NF.**

Conclusion:

All 17 relations in the schema are in 3NF because:

1. Each non-key attribute is functionally dependent on the primary key.
2. There are no transitive dependencies.
3. Derived attributes are handled appropriately to avoid update anomalies.

Sample Queries:

1. Total Sales for the Store in January 2026.
2. Most Popular Item by Quantity Sold.
3. Best-Selling Items in a specific 30 Days.
4. Total Revenue per Store and Date.
5. Customers Who Have Made Orders.
6. Store with the Most Sales in 2026 (For Store_ID = 1).
7. Employees Who Have Processed the Most Orders.
8. Products That Are Low in Stock (Quantity <= 20).
9. Total Products Sold by Customer in the Last 6 Months.
10. Average Order Value for Each Customer.
11. Total Supply Orders Count Within a Date Range.
12. Total Purchase Amount Within a Date Range.
13. Total Purchase Amount per Supplier.
14. Supply Orders Count per Supplier.
15. Daily Supply Orders Summary.
16. Top Purchased Items by Quantity.
17. Top Purchased Items by Total Cost.
18. Supply Orders Detailed View supply order Supplier + Employee + Warehouse.
19. Supply Order Lines Detailed View (Header + Items).
20. Current Warehouse Stock per Item.
21. Total Quantity in Warehouse.
22. Warehouse Capacity vs Current Stock.

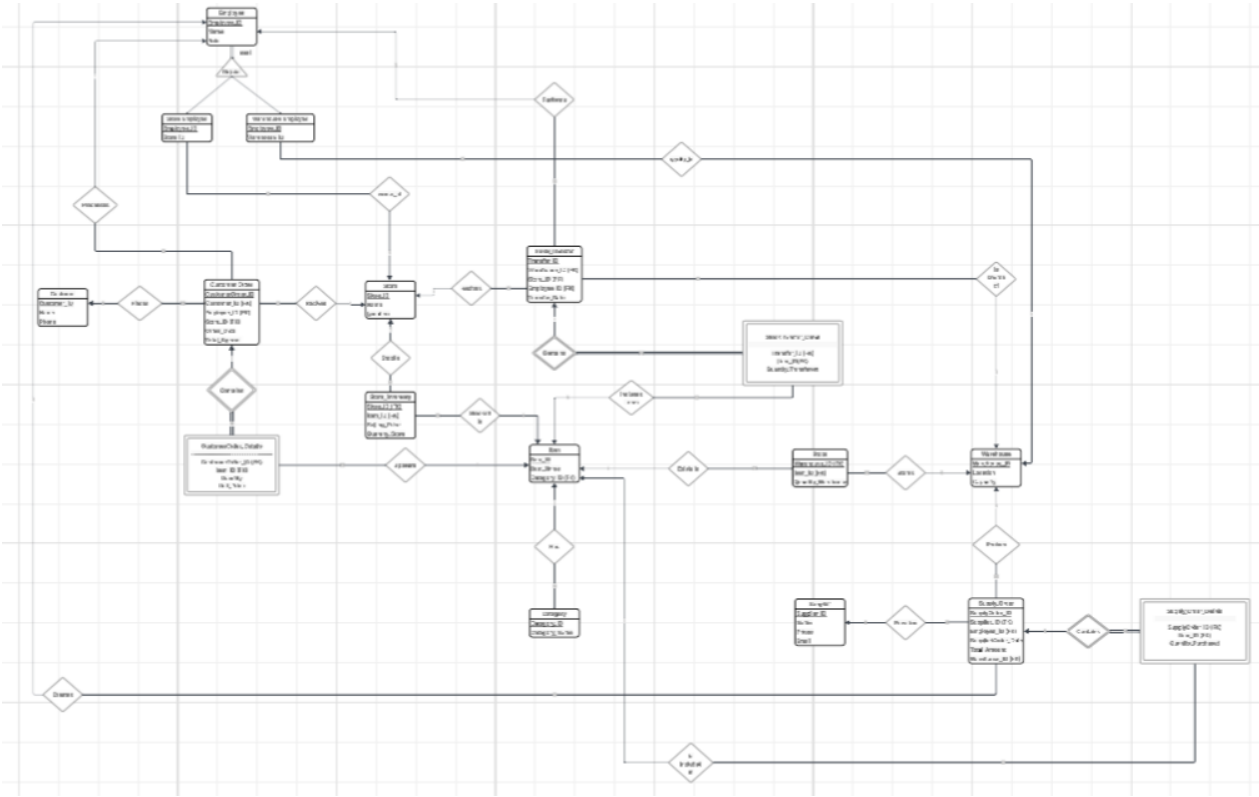
Technology Stack:

Database Management System: MySQL

Backend Development: Python

Frontend Development: Tkinter (Python Library)

Entity-Relationship Diagram (ERD):



Note: Due to the large number of entities, this is the best photo quality we were able to get in Word.
For a clearer view, here is the diagram link directs you to the project diagram. Everything is detailed in the documentation above.

Diagram Link: **https://lucid.app/lucidchart/0f377de7-e901-4cb3-a530-fcd1c3b3d062/edit?viewport_loc=-6176%2C-4150%2C9278%2C3390%2C0_0&invitationId=inv_d8fb47e9-4235-4acf-a6d9-5a6e502cb64c**